Name: Apurba Koirala

22BCE3799

Deep Learning Lab Assessment 3

Guided by: Dr. Sharmila C

1. Perform object detection using Transfer Learning with a pre-trained CNN architecture (e.g., VGG16, ResNet, or InceptionNet). Fine-tune the selected model on any dataset and evaluate its performance.

```
from tensorflow.keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
from tensorflow.keras.utils import to_categorical

x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

from tensorflow.keras.applications import VGG16

base_model = VGG16(weights='imagenet', include_top=False, input_shape=(32,
32, 3))
base_model.trainable = False

for layer in base_model.layers[-4:]:
    layer.trainable = True

from tensorflow.keras import layers, models

model = models.Sequential([
    base_model,
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

from tensorflow.keras.optimizers import Adam
model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
```

```
                    validation_data=(x_test, y_test),
                    epochs=10,
                    batch_size=64)
```

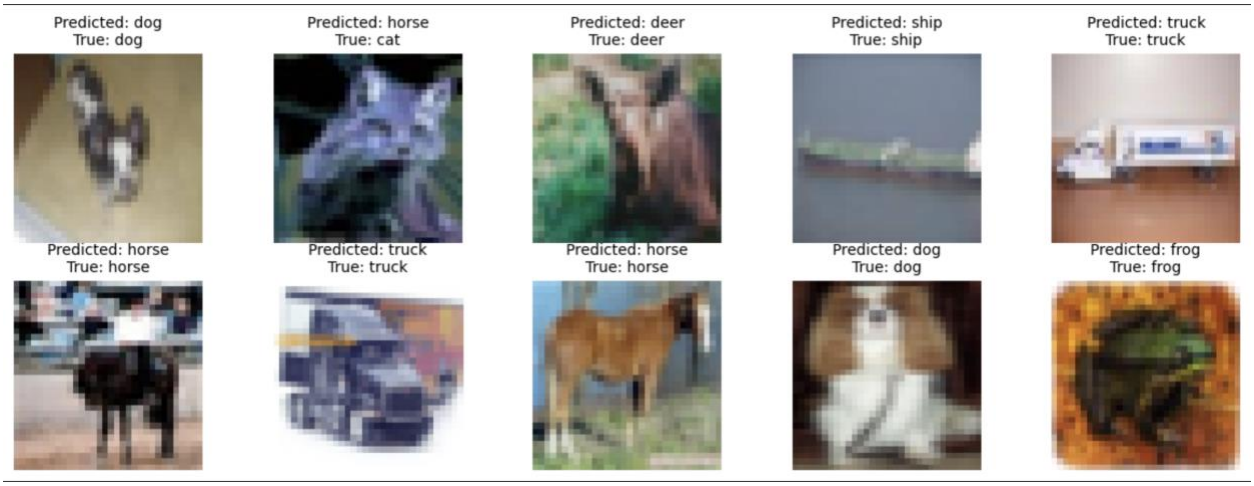
```
Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 24s 26ms/step - accuracy: 0.5058 - loss: 1.4154 - val_accuracy: 0.7014 - val_loss: 0.8556
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.7133 - loss: 0.8384 - val_accuracy: 0.7296 - val_loss: 0.7820
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 15s 20ms/step - accuracy: 0.7642 - loss: 0.6895 - val_accuracy: 0.7412 - val_loss: 0.7656
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 15s 19ms/step - accuracy: 0.8035 - loss: 0.5708 - val_accuracy: 0.7499 - val_loss: 0.7334
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.8413 - loss: 0.4681 - val_accuracy: 0.7562 - val_loss: 0.7402
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.8733 - loss: 0.3750 - val_accuracy: 0.7567 - val_loss: 0.8039
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 15s 20ms/step - accuracy: 0.8979 - loss: 0.2972 - val_accuracy: 0.7526 - val_loss: 0.8735
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 15s 20ms/step - accuracy: 0.9226 - loss: 0.2274 - val_accuracy: 0.7443 - val_loss: 0.9480
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 15s 20ms/step - accuracy: 0.9480 - loss: 0.1598 - val_accuracy: 0.7485 - val_loss: 1.0028
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 15s 20ms/step - accuracy: 0.9574 - loss: 0.1262 - val_accuracy: 0.7491 - val_loss: 1.0968
```

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Test Accuracy: {test_acc:.4f}")
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
'frog', 'horse', 'ship', 'truck']

num_images = 10
indices = np.random.choice(len(x_test), num_images, replace=False)
sample_images = x_test[indices]
sample_labels = y_test[indices]

predictions = model.predict(sample_images)

plt.figure(figsize=(15, 5))
for i in range(num_images):
    plt.subplot(2, 5, i + 1)
    plt.imshow(sample_images[i])
    true_label = class_names[np.argmax(sample_labels[i])]
    predicted_label = class_names[np.argmax(predictions[i])]
    plt.title(f"Predicted: {predicted_label}\nTrue: {true_label}",
fontsize=10)
    plt.axis('off')
plt.show()
```



2. Implement an Autoencoder for image denoising using the CIFAR-10 dataset. Add Gaussian noise to the input images and train the autoencoder to reconstruct clean images. Evaluate the performance by visualizing the original, noisy and reconstructed images.

```
      (x_train, y_train), (x_test, y_test) = cifar10.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
noise_factor = 0.3
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0,
scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0,
scale=1.0, size=x_test.shape)
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

from tensorflow.keras import layers, models

input_img = layers.Input(shape=(32, 32, 3))
x = layers.Conv2D(32, (3, 3), activation="relu",
padding="same")(input_img)
x = layers.MaxPooling2D((2, 2), padding="same")(x)
x = layers.Conv2D(64, (3, 3), activation="relu", padding="same")(x)
encoded = layers.MaxPooling2D((2, 2), padding="same")(x)
x = layers.Conv2D(64, (3, 3), activation="relu", padding="same")(encoded)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(x)
x = layers.UpSampling2D((2, 2))(x)
```

```
decoded = layers.Conv2D(3, (3, 3), activation="sigmoid",
padding="same")(x)

autoencoder = models.Model(input_img, decoded)
autoencoder.compile(optimizer="adam", loss="mse")
autoencoder.summary()
```

Model: "functional_3"

Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, 32, 32, 3)	0
conv2d_5 (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_6 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_7 (Conv2D)	(None, 8, 8, 64)	36,928
up_sampling2d_2 (UpSampling2D)	(None, 16, 16, 64)	0
conv2d_8 (Conv2D)	(None, 16, 16, 32)	18,464
up_sampling2d_3 (UpSampling2D)	(None, 32, 32, 32)	0
conv2d_9 (Conv2D)	(None, 32, 32, 3)	867

Total params: 75,651 (295.51 KB)
Trainable params: 75,651 (295.51 KB)
Non-trainable params: 0 (0.00 B)

```
history = autoencoder.fit(
    x_train_noisy, x_train,
    epochs=30,
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)
```

Last 10 epochs:

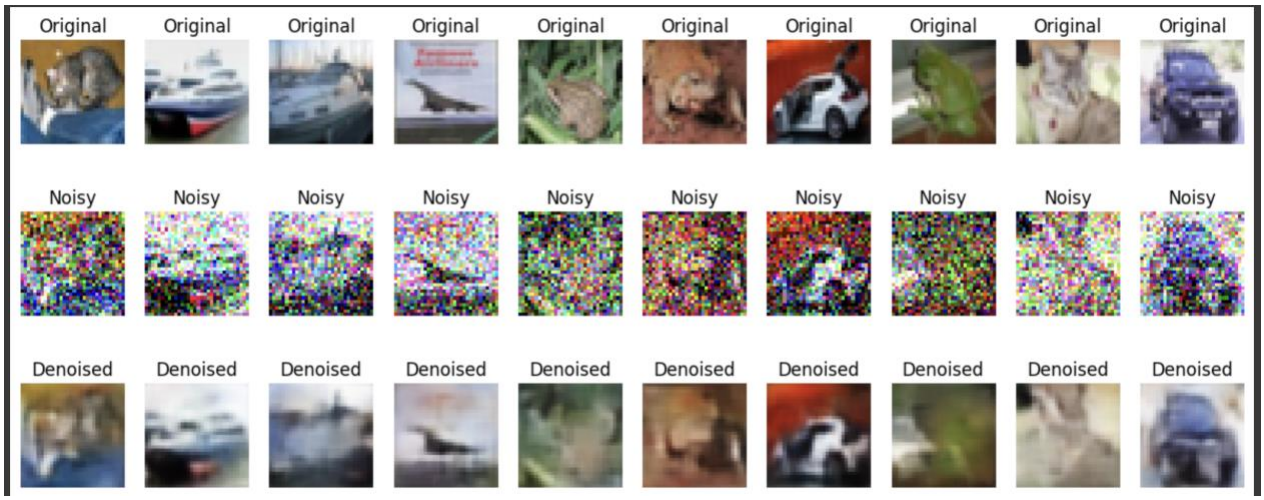
```
Epoch 20/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0063 - val_loss: 0.0063
Epoch 21/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0063 - val_loss: 0.0063
Epoch 22/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0063 - val_loss: 0.0063
Epoch 23/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0063 - val_loss: 0.0062
Epoch 24/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0062 - val_loss: 0.0063
Epoch 25/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0062 - val_loss: 0.0062
Epoch 26/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0062 - val_loss: 0.0062
Epoch 27/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0062 - val_loss: 0.0062
Epoch 28/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0062 - val_loss: 0.0061
Epoch 29/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0061 - val_loss: 0.0062
Epoch 30/30
391/391 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0061 - val_loss: 0.0061
```

```python
decoded_imgs = autoencoder.predict(x_test_noisy)
n = 10
plt.figure(figsize=(15, 6))
for i in range(n):

    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test[i])
    plt.title("Original")
    plt.axis("off")

    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(x_test_noisy[i])
    plt.title("Noisy")
    plt.axis("off")

    ax = plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(decoded_imgs[i])
    plt.title("Denoised")
    plt.axis("off")
plt.show()
```


Original
Original
Original
Original
Original
Original
Original
Original
Original
Original
Noisy
Noisy
Noisy
Noisy
Noisy
Noisy
Noisy
Noisy
Noisy
Noisy
Denoised
Denoised
Denoised
Denoised
Denoised
Denoised
Denoised
Denoised
Denoised
Denoised